

JavaScript Interview Preparation Guide

Fundamentals

1. Function Declaration vs. Expression vs. Arrow Function

Function Declaration: `function sum(a, b) { return a + b; }`. Hoisted entirely to the top of its scope, meaning it can be called before it is defined. It has its own `this`, an `arguments` object, and can be used as a constructor (with `new`).

Function Expression: `const sum = function(a, b) { return a + b; };`. Only the variable declaration is hoisted (if using `var`, it's undefined; if `let/const`, it's in the Temporal Dead Zone). Cannot be called before initialization. Supports `this`, `arguments`, and `new`.

Arrow Function: `const sum = (a, b) => a + b;`. Not hoisted (acts like an expression). It does *not* have its own `this`; it lexically inherits `this` from the surrounding scope. It lacks an `arguments` object (use rest parameters instead) and cannot be used as a constructor (throws an error with `new`).

2. First-Class and Higher-Order Functions

First-class functions: In JavaScript, functions are treated as variables. They can be assigned to variables, passed as arguments to other functions, or returned from other functions.

Higher-order functions: A function that takes another function as an argument (callback) or returns a function. Examples include `Array.prototype.map`, `filter`, and `reduce`.

They enable callbacks by allowing functions to be passed into asynchronous operations (like `setTimeout` or API calls) to be executed later. They support reusable abstractions by letting developers write generic logic (like traversing an array) while passing in specific logic (like transforming data) as a function, forming the basis of functional programming in JS.

3. Closures and Private State

A closure is the combination of a function bundled together (enclosed) with references to its surrounding state (the lexical environment). In JS, closures are created every time a function is created, at function creation time.

When an outer function returns an inner function, the inner function maintains a reference to the outer function's variable environment, preventing those variables from being garbage collected. This is how it "remembers" variables.

Private State Example:

```
function createCounter() {
  let count = 0; // Private variable
  return {
    increment: () => ++count,
    getCount: () => count
  };
}
const counter = createCounter();
// count is inaccessible directly; only via increment/getCount.
```

4. Currying vs. Partial Application

Currying transforms a function taking multiple arguments into a sequence of functions, each taking a single argument. `f(a, b, c)` becomes `f(a)(b)(c)`.

Partial Application fixes a specific number of arguments to a function, producing another function of smaller arity. `f(a, b, c)` partially applied with `a` becomes `f(b, c)`.

```
// Currying
const curry3 = fn => a => b => c => fn(a, b, c);
const add3 = curry3((x, y, z) => x + y + z);
console.log(add3(1)(2)(3)); // 6

// Partial Application
const partialAdd = (fn, ...fixed) => (...rest) => fn(...fixed, ...rest);
const add1 = partialAdd((x, y, z) => x + y + z, 1);
console.log(add1(2, 3)); // 6
```

5. Pure vs. Impure Functions

Pure Functions: Given the same input, they always return the same output. They produce zero side effects (no DOM mutation, no network requests, no modifying global state or input arguments).

Impure Functions: May return different results for the same inputs (e.g., uses `Math.random()` or reads from a global variable) or cause side effects.

Pure functions improve predictability (easy to trace logic without hidden dependencies), testing (no need for complex mocking of state), and memoization (results can be safely cached based purely on input arguments).

Intermediate

1. `call()`, `apply()`, and `bind()`

All three are used to manually set the `this` context of a regular function.

- `call(thisArg, arg1, arg2, ...)` : Invokes the function immediately, passing arguments individually.
- `apply(thisArg, [argsArray])` : Invokes the function immediately, passing arguments as an array.
- `bind(thisArg, arg1, ...)` : Returns a *new* function with `this` permanently bound. It does not invoke the function immediately.

Arrow Functions: None of these methods can alter an arrow function's `this`. An arrow function permanently resolves its `this` lexically (where it was defined in the source code) at creation time, bypassing execution context binding.

2. Default, Rest, Spread, and arguments

Default parameters: Allow named parameters to be initialized with default values if no value or `undefined` is passed. (Note: passing `null` does *not* trigger the default parameter).

Rest parameters (`...args`): Collects multiple trailing arguments into a real Array. It must be the last parameter. It is the modern replacement for `arguments`.

Spread syntax (`...iterable`): Expands an array or iterable into individual elements. Used in function calls or array/object literals.

arguments object: An array-like (but not a real Array) object available inside non-arrow functions containing all passed arguments. It tracks missing arguments as `undefined` but does not ignore `null`.

3. Function Composition

Function composition is the process of combining two or more functions to produce a new function. Executing the combined function routes the output of one function to the input of the next. `f(g(x))`.

```
const compose = (...fns) => x =>
  fns.reduceRight((acc, fn) => fn(acc), x);

// Usage:
const add2 = x => x + 2;
const multiply3 = x => x * 3;
const addThenMultiply = compose(multiply3, add2); // Right-to-Left
console.log(addThenMultiply(5)); // (5 + 2) * 3 = 21
```

4. Function Factory using Closures

A function factory is a function that creates and returns other functions, often using closures to configure the returned function with specific, private parameters.

```
// Factory creating configurable validators
function createValidator(minLength) {
  return function(stringToTest) {
    return stringToTest.length >= minLength;
  };
}

const isPasswordValid = createValidator(8);
const isPinValid = createValidator(4);

console.log(isPasswordValid("pass")); // false
console.log(isPinValid("1234")); // true
```

The inner function retains access to `minLength` via closure, effectively baking configuration into the newly generated function.

5. Recursion vs. Iteration

Iteration (loops): Uses control structures (for, while). Usually more memory-efficient as it runs within a single stack frame.

Recursion: A function calls itself. Can yield cleaner, more declarative code (especially for trees/graphs).

- **Call-stack limits:** Every recursive call adds a frame to the call stack. Deep recursion can exceed the engine's maximum stack size, causing a "Maximum call stack size exceeded" error. Iteration doesn't face this limit.
- **Time Complexity:** Often similar, but recursion can be slower due to the overhead of function calls and frame allocation.
- **Space Complexity:** Iteration is generally $O(1)$ auxiliary space. Recursion is $O(n)$ due to stack frames, unless Tail Call Optimization (TCO) is supported by the engine.

Advanced

1. Generic memoize() implementation

Memoization caches the results of expensive function calls.

```
function memoize(fn) {
  const cache = new Map();
  return function(...args) {
    // Stringify handles primitives well, but struggles with identical objects.
    // A robust approach creates a compound key or uses nested Maps.
    const key = JSON.stringify(args);

    if (cache.has(key)) {
      return cache.get(key);
    }
    const result = fn.apply(this, args);

    // Cache invalidation/Unbounded growth:
    // To prevent unbounded growth, implement an LRU (Least Recently Used) cache
    // or limit Map size, evicting the oldest key when size > LIMIT.
    if (cache.size > 1000) {
      const firstKey = cache.keys().next().value;
      cache.delete(firstKey);
    }

    cache.set(key, result);
    return result;
  };
}
```

Object Arguments: `JSON.stringify` is slow and fails on circular references. Better implementations use `WeakMap` for object references to allow garbage collection and avoid memory leaks.

2. Debounce and Throttle

Debounce: Delays function execution until a certain amount of time has passed since the *last* call. Good for search inputs (wait until user stops typing).

```
function debounce(fn, delay) {
  let timeoutId;
  return function(...args) {
    clearTimeout(timeoutId);
    timeoutId = setTimeout(() => fn.apply(this, args), delay);
  };
}
```

Throttle: Ensures a function is called at most once in a specified time period, even if triggered multiple times. Good for window resizing or scroll events.

```
function throttle(fn, limit) {
  let inThrottle;
  return function(...args) {
    if (!inThrottle) {
      fn.apply(this, args);
      inThrottle = true;
      setTimeout(() => inThrottle = false, limit);
    }
  };
}
```

3. Parameter Initialization in ECMAScript

In ES6, parameters are initialized from left to right in their own intermediate scope (between the outer scope and the function body). This is why a parameter can reference parameters to its left (e.g., `b = a + 1`). If it references a parameter to its right, it hits a Temporal Dead Zone (TDZ) and throws a `ReferenceError` because that right-hand parameter has not been initialized yet.

4. Closures and Memory Retention

Closures retain references to their outer lexical environments. If a closure is attached to a long-lived object (like a DOM event listener, a global `setInterval`, or a globally cached function), it prevents the garbage collector from freeing the memory of *all* variables in its outer scope.

For example, if an event listener references a large data object, and the DOM element is removed but the listener isn't explicitly removed (`removeEventListener`), the closure and the large object remain in memory, causing a leak.

5. Generic Function Wrapper

To wrap a function safely without mutating its behavior, you must preserve `this`, arguments, return values, and exceptions.

```
function withLogging(fn) {
  return function(...args) {
    console.log(`Calling with arguments: ${args}`);
    try {
      // Preserve `this` and arguments
      const result = fn.apply(this, args);
      console.log(`Returned: ${result}`);
      return result; // Preserve return value
    } catch (err) {
      console.error(`Threw error: ${err}`);
      throw err; // Preserve exception behavior
    }
  };
}
```

Output-Based Questions

16. Hoisting and Initialization

```
console.log(typeof greet); // "function"
console.log(typeof sayHello); // "undefined"
function greet() {}
var sayHello = function () {};
```

Explanation: `greet` is a function declaration and is fully hoisted to the top, so its type is immediately "function". `sayHello` is declared with `var`, so its declaration is hoisted but *not* its assignment. It exists in memory as `undefined` until the engine reaches the assignment line.

17. Callbacks and Loop Scopes

```
for (var i = 0; i < 3; i++) { setTimeout(() => console.log(i), 0); }
// Output: 3, 3, 3
for (let j = 0; j < 3; j++) { setTimeout(() => console.log(j), 0); }
// Output: 0, 1, 2
```

Explanation: `var i` is function/globally scoped. By the time the async `setTimeout` callbacks execute, the loop has finished and `i` is 3. All callbacks share the same `i`. `let j` is block-scoped. In each iteration, a fresh block scope is created for `j`. Each callback captures its own unique instance of `j` via closure.

18. Lexical this and Arrow Functions

```
"use strict";
const user = {
  name: "Alex",
  regular() { return this.name; },
  arrow: () => this.name
};
console.log(user.regular()); // "Alex"
console.log(user.arrow()); // Cannot read properties of undefined (or undefined/empty
depending on environment)
```

Explanation: `regular()` is invoked as a method on `user`, so its `this` is dynamically bound to `user`. `arrow()` is an arrow function. It lexically inherits `this` from the scope where `user` was defined (the global scope). In strict mode, global `this` is `undefined`, causing an error (or referencing `global/Window` in non-strict browsers).

19. Default Parameters and null

```
function test(a = 1, b = a + 2, c = b + 3) {
  return [a, b, c];
}
console.log(test()); // [1, 3, 6]
console.log(test(10, undefined, 20)); // [10, 12, 20]
console.log(test(null)); // [null, 2, 5]
```

Explanation:

Call 1: All use defaults.

Call 2: `a` is 10, `b` hits `undefined` so it uses the default ($10 + 2 = 12$), `c` is 20.

Call 3: `null` is a valid value, so `a` becomes `null`. In `math(null + 2)`, `null` coerces to `0`, so `b = 2` and `c = 5`.

20. Closures and Independent Instances

```
const a = createCounter();
const b = createCounter();
a.increment(); a.increment(); b.increment();
console.log(a.getCount()); // 2
console.log(b.getCount()); // 1
```

Explanation: Every time `createCounter()` is called, a brand new execution context is created with its own independent `count` variable. The closures returned to `a` point to a different `count` than the closures returned to `b`. They maintain entirely separate state.